Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 54 / 54 testcases passed

yousab21 submitted at Apr 25, 2026 19:04

Analysis | Solution

Runtime

0 ms | Beats 100.00%

Memory

9.90 MB | Beats 10.37%

Code | C++

```cpp
class Solution {
public:
    bool isAnagram(string s, string t) {
        // if stings are not the same size return immidiatly
        if(s.size() != t.size()){
            return false;
        }

```

View more

More challenges

- 266. Palindrome Permutation
- 438. Find All Anagrams in a String
- 2273. Find Resultant Array After Removing Anagrams

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```cpp
class Solution {
public:
    bool isAnagram(string s, string t) {
        // if stings are not the same size return immidiatly
        if(s.size() != t.size()){
            return false;
        }

        //we use a frequancy array here because letters can be in any order
        //we reserve 26 blocks beacue the alphabet has 26 leteters, all blocks are initially reserved as 0
        vector<int> freq(26, 0);

        // this is the main logic of the problem we make it so that one sting addes +ive freq to letters
        // and the other addes -ive freq
        // in an anagram those cancel out each other but if there were extra or missing letters in one string
        // it will remain in the frequancy array
        for (int i = 0; i < s.size(); i++) {
        freq[s[i] - 'a']++;
        freq[t[i] - 'a']--;
        }

        // if anything remains in the freq array return false
        for(int i =0 ; i < freq.size() ; i++){
            if(freq[i] != 0){
                return false;
            }
        }

        return true;
    }
};
```

Saved Ln 29, Col 21

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 128 / 128 testcases passed

yousab21 submitted at Apr 25, 2026 20:33

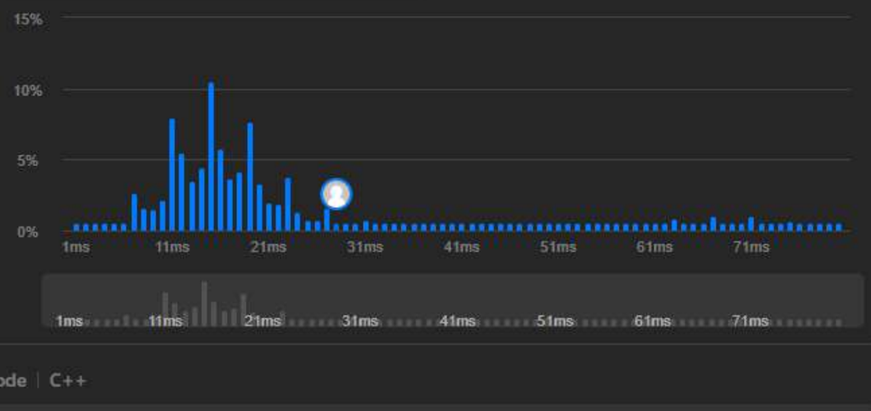
Analysis | Solution

Runtime

28 ms | Beats 22.26%

Memory

31.44 MB | Beats 15.55%



Code | C++

```cpp
// here we also use a frequancy array for the same reasons
class Solution {
public:

    // this function just builds a frequancy array for a passed string
    // i just pulled it from the main logic to clear up some nesting
    vector<int> getFreq(string& str){
        vector<int> freq(26, 0);
```

View more

More challenges

- 249. Group Shifted Strings
- 2273. Find Resultant Array After Removing Anagrams
- 2514. Count Anagrams

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```cpp
// here we also use a frequancy array for the same reasons
class Solution {
public:

    // this function just builds a frequancy array for a passed string
    // i just pulled it from the main logic to clear up some nesting
    vector<int> getFreq(string& str){
        vector<int> freq(26, 0);
        // same logic as last problem but with just only addition this time
        for(int i = 0 ; i < str.size() ; i++){
            freq[str[i] - 'a']++;
        }

        return freq;
    }

    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        //there is some pretty big cases here so using normals arrays is
        //to slow for the amout of searching and comparasion i was doing here
        //the solution is to use a map as searching a map has O(1)
        // the key array here acts as "commonFreqArray" and the value array acts as "allStringsThatAgreeWithKey"
        //this have the advatage of anagrams being automatcly grouped together as they will have the same key (freq array)
        map<vector<int>, vector<string>> groups;

        // first i loop over all stings
        for(int i = 0; i < strs.size(); i++){
            //i get the frequancy array of the sting iam on
            vector<int> freq = getFreq(strs[i]);

            // and i use that frequancy array as a key in the map
            // if the kay doesnt exsist it is made automaticly and i push the string as the first element of the
            // if the key does alreay exists the string gets pushed to the valueArray
            groups[freq].push_back(strs[i]);
        }
        //at this point we have all anagrams grouped together, problem is it is in a map and
        // we need to return a vector of vectors
        // so we need to "parse" the map as a vector of vectors here is how i do this

        //this will act as my result vactor
        vector<vector<string>> anagrams;
        //i just iterate on all keys and push the value vector to the result one
        // maps in c++ use stl-iteratrs not indexes to this is why ot looks weird
        for(auto i = groups.begin(); i != groups.end(); i++){
        //(iterator->first gives key )
        //(iterator->second gives you the value)
        anagrams.push_back(i->second);
        }

        return anagrams;
    }
```

Saved Ln 43, Col 58

Testcase | Test Result